

GalaxyGate + RunPod track

Scan for the written guide

<https://github.com/GalaxyGate/philly-hackathon>



Before you start

- A laptop you can install software on
- A GitHub account
- An email inbox you can open now
- If you are a team, one teammate does Part 1 and shares the resulting URL and repository with the others
- If you are following this from Discord and not at the venue, post in the Discord at <https://discord.com/invite/HP4BhW3hnp> and an organizer will send you the GalaxyGate coupon code and your RunPod credit link.

Step 1. Make a folder, pick one tool

- Create an empty folder on your Desktop named `hackathon`. Everything below happens in that folder.
- Pick one tool. If you have none, pick Cursor.
- **Cursor.** Download from <https://cursor.com/download>, sign in, then File, Open Folder, and choose `hackathon`. If you received a Cursor credit code card at the GalaxyGate table, redeem it at the link on the card before you continue.
- **Claude Code.** Needs a paid Claude plan; if you do not have one, pick Cursor.
- **Codex.** Needs a paid ChatGPT plan; if you do not have one, pick Cursor.

Step 1, continued. Open a terminal

- **Windows.** Right-click the `hackathon` folder, choose Open in Terminal. Also install Git for Windows from <https://git-scm.com/downloads/win> once with its default options; your coding agent runs the setup commands through it.
- **macOS.** Open Terminal from Spotlight, type `cd` with a space, drag the `hackathon` folder onto the Terminal window, press Enter. The prompt now ends in `hackathon`.
- Terminal commands in this guide are typed in a terminal. Prompts are pasted into the tool's chat.

Step 1, continued. Install Claude Code

In a terminal opened in `hackathon` run the installer. On macOS:

```
curl -fsSL https://claude.ai/install.sh | bash
```

On Windows in PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

(your prompt starts with `PS` in PowerShell)

Open a new terminal in `hackathon`, run `claude`, and sign in when it asks.

Step 1, continued. Install Codex

In a terminal opened in `hackathon` run the installer. On macOS:

```
curl -fsSL https://chatgpt.com/codex/install.sh | sh
```

On Windows in PowerShell:

```
irm https://chatgpt.com/codex/install.ps1 | iex
```

Open a new terminal in `hackathon`, run `codex`, and sign in when it asks.

Step 2. GalaxyGate account

1. Open <https://dash.galaxygate.net/-/register>. If the coupon box is empty, type the coupon code from the GalaxyGate table or from the organizer on Discord.
2. Open the verification email and click the link. Check your spam folder if it is not there within a minute.
3. Sign in at <https://dash.galaxygate.net>. You should see one workspace and no servers.

If you already had a GalaxyGate account before the event, sign in and ask at the GalaxyGate table to apply the coupon.

Coupon:

 Optional. Applies credit to your new account.

Step 3. RunPod account and one API key

1. Sign up at <https://console.runpod.io/signup>.
2. Open your RunPod credit link and redeem it. Your balance should read \$15.
3. In the console open Settings, expand API Keys, click Create API Key. Name it `hackathon-app`, choose the permission `All`, create it, and copy the key now. RunPod shows it only once. You paste it into the prompt in step 6.

This key can spend your credit. It goes into your app's environment on your server, never into your code or your repository. Revoke it on the same page after the event.

Step 4. MCP servers in Cursor

Both servers sign you in through a browser window. There is no key to paste for this step.

In Cursor's Explorer sidebar click New File, type `.cursor/mcp.json` as the whole file name, press Enter, and paste this. Do not try to create the `.cursor` folder in File Explorer or Finder; they refuse names starting with a dot. If the file already exists, add the two entries inside its `mcpServers` object instead of replacing it.

```
{
  "mcpServers": {
    "galaxygate": { "url": "https://mcp.galaxygate.net/mcp" },
    "runpod": { "url": "https://mcp.getrunpod.io/" }
  }
}
```

Step 4. Cursor sign-in

1. Save and restart Cursor.
2. Open Customize in the sidebar, then MCP.
3. Each of the two servers shows Needs login. Click it, approve in the browser tab that opens, come back.
4. Check the toggle next to each server is on.

Step 4. MCP servers in Claude Code

Both servers sign you in through a browser window. There is no key to paste for this step. In a terminal opened in `hackathon`:

```
claude mcp add --transport http --scope user galaxygate https://mcp.galaxygate.net/mcp
claude mcp add --transport http --scope user runpod https://mcp.getrunpod.io/
```

Then start `claude`, type `/mcp`, and complete the sign-in for each server.

Step 4. MCP servers in Codex

Both servers sign you in through a browser window. There is no key to paste for this step. In a terminal opened in `hackathon`:

```
codex mcp add galaxygate --url https://mcp.galaxygate.net/mcp
codex mcp login galaxygate
codex mcp add runpod --url https://mcp.getrunpod.io/
codex mcp login runpod
```

Step 4. Start Codex for this project

Codex blocks network access inside its sandbox by default, and the setup prompt needs SSH. Start Codex for this project with:

```
codex --sandbox workspace-write -c sandbox_workspace_write.network_access=true
```

Codex will ask permission the first time the agent writes the SSH key and connects to your server; approve those requests. Do not turn the sandbox off.

Step 4. Check the connection

```
Call the GalaxyGate list_workspaces tool and the RunPod  
list-endpoints tool and show me both results.
```

- You should see one GalaxyGate workspace with your name on it, and a RunPod endpoint list, which may be empty
- If either call is denied, redo the sign-in for that server

Step 5. Windows only: SSH

This check is for your own terminal; your agent runs its commands through Git Bash, which brings its own SSH.

1. In a terminal opened in `hackathon` run `ssh -V`. If it prints a version, skip ahead.
2. Otherwise open PowerShell as administrator (Start, type PowerShell, right-click, Run as administrator), run this, wait for it to finish, close that window, and run `ssh -V` in your normal terminal again.

```
Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0
```

(the name contains four tildes)

macOS and Linux already have SSH.

Step 6. Create your server: paste this

```
TEAM_NAME=yourteam  
RUNPOD_API_KEY=rpa_paste-your-key-here
```

Read <https://raw.githubusercontent.com/GalaxyGate/philly-hackathon/main/AGENT.md>
and follow it step by step. Stop and tell me if the page does not load.

Replace `yourteam` with your team name: 3 to 32 characters, lowercase letters, digits, and hyphens only, starting and ending with a letter or digit. Replace the key.

If your tool cannot open web pages, open

<https://github.com/GalaxyGate/philly-hackathon/blob/main/AGENT.md> yourself, copy its whole text, and paste it under the two lines instead.

Done when

- The agent reports that both checks passed and gives you your URL, and that URL opens in your browser and roasts a public GitHub repository when you paste one
- It writes `HACKATHON.md` in your folder with every id you will need

GitHub repository URL

`https://github.com/encode/httpx`

Roast

Writing the roast

Build: the one rule

- Your app, its data, and your RunPod key live on your GalaxyGate server.
- Your app's backend calls RunPod over HTTPS.
- Never call RunPod from browser JavaScript, and never put the key in your repository.
- Your RunPod key is now in your agent's chat history and in the app's environment on your server, where anyone in your GalaxyGate workspace can see it. Keep the workspace to your team and revoke the key after the event.
- Your SSH key is in the `.hackathon` folder inside your project; it is listed in `.gitignore`, so it stays out of your repository.

Fork and clone

The demo running on your server is `demos/roast` in <https://github.com/GalaxyGate/philly-hackathon>. Fork that repository into your GitHub account, then tell your agent:

```
Clone https://github.com/<your GitHub username>/philly-hackathon into this folder.
```

Edit whatever you like, or add a new app folder next to the demos.

As shipped, the demo forgets its roast when the page reloads; keeping results across a reload is worth 5 scoring points.

Redeploy your changes

To put your changes on your server, tell your agent:

```
Follow the redeploy section of AGENT.md for demos/roast.
```

The agent copies the folder to your server over SSH, builds the image there into a registry that runs on the server, and switches the app to it. On Windows this needs Git for Windows.

Ask `Show me the app logs` when something is wrong, and `Restart the app` to restart it.

Use a GPU model: ready-made

```
Add a route to my app that generates an image with RunPod's
Flux Schnell public endpoint using the RUNPOD_API_KEY from
the environment, saves the image under /data, and shows it on
a page.
```

The endpoint is `https://api.runpod.ai/v2/black-forest-labs-flux-1-schnell/runsync`; it takes `{"input":{"prompt":"...", "width":768, "height":768}}` with the key as a bearer token, width and height between 256 and 1536 and divisible by 64, and returns the image URL at `output.image_url`.

Other ready-made models, including Qwen for text, are listed at <https://docs.runpod.io/public-endpoints/overview>.

Use a GPU model: your Hugging Face model

```
Using the RunPod MCP, call list-endpoints and stop if an
endpoint named hackathon-llm exists. Otherwise call
deploy-hub-repo with repo runpod-workers/worker-vllm, name
hackathon-llm, gpuIds ADA_24, workersMin 0, workersMax 1,
idleTimeout 60, and env
MODEL_NAME=Qwen/Qwen2.5-Coder-7B-Instruct and
MAX_MODEL_LEN=8192. Then send one chat completion to it and
show me the reply and the endpoint id.
```

Your app then calls `https://api.runpod.ai/v2/<ENDPOINT_ID>/openai/v1/chat/completions` with your key and that model name. The first request after a quiet period waits while the model loads; keep your app's timeout above two minutes.

Spending

- The demo app caps itself at 25 generations per hour so a stranger cannot drain your credit.
- The cap is the `MAX_GENERATIONS_PER_HOUR` environment variable. Keep a cap in anything you build.
- The roast demo also makes two GitHub API calls per roast, and GitHub allows 60 per hour per server without a token.
- If you need more, ask your agent to add a `GITHUB_TOKEN` environment variable holding a fine-grained GitHub token with public read access, keeping every existing variable.

Eligibility

- Your GalaxyGate server and app were created during the build period. The organizers' starter code and RunPod's public models are allowed; say what you reused.
- Fabricated evidence, including a faked RunPod call, removes the project from this track.

Submit on Devpost

- Submit at <https://coffee-and-code-agent.devpost.com/> before the deadline shown there, and select the GalaxyGate prize.
- Your app URL, GalaxyGate instance id and app id (all in `HACKATHON.md`), a screenshot of your workspace's Workflows page in the panel, and one exact input to try.
- Your RunPod endpoint id, or the public model you used, plus one request record: a screenshot of the request in your RunPod console, or the `id` field from one RunPod response.
- Your repository, the commit you started from, the commit you submitted, and a short list of what you changed or built.
- A video under three minutes and a README that lets a stranger run it. Check both open in a browser where you are signed out.
- Do not include environment screens, API keys, private keys, or credit links in anything you submit.
- Check your RunPod balance in the console before you submit.

Scoring, 100 points

- **Live app, 20.** The URL opens (5), a judge's fresh input succeeds (10), and the same result is still on screen after the judge reloads the page (5).
- **RunPod integration, 20.** A judge's fresh input produces a request visible in your RunPod console or a RunPod response `id` you show the judge (10), and your app visibly uses the model's result (10).
- **Agent-operated infrastructure, 15.** Your GalaxyGate workflow history shows an `instance.provision` (5), an `app.provision` (5), and either an `app.power` row or more `app.deploy` rows than `app.provision` rows (5), because each provision brings exactly one `app.deploy` of its own, every later redeploy adds an `app.deploy`, and every restart adds an `app.power`. All rows must fall inside the build period. The panel shows them on your workspace's Workflows page; screenshot it.

Scoring, continued

- **Agentic usefulness, 25.** A stated user task with a clear success condition (5), a page or log the judge can open that shows each model decision and each tool result and grows when the judge submits their own input (10), success on that judge-supplied input (5), and one honestly demonstrated limitation (5). A single fixed model call earns no action points.
- **Reliability, safety, cost, 15.** Secrets kept out of the repository, logs, and screenshots (5), paid routes protected by a cap or login (5), a slow start or upstream error handled visibly (5).
- **Submission, 5.** Video (2), README (2), the disclosures above (1).
- Ties break on agentic usefulness, then reliability. If your app is down when judges test it, a timestamped end-to-end recording counts for everything except the live-app points.

When something fails

- **An MCP server shows Needs login or a tool call is denied.** Redo that server's sign-in: Cursor in Customize, MCP; Claude Code with `/mcp`; Codex with `codex mcp login <name>`.
- **The agent says the server never became ready.** Ask it to run `get_instance` and show the state, and `cloud-init status --long` over SSH. Bring both to the GalaxyGate table.
- **The first app deploy sticks or fails.** The agent instructions recover once on their own. If it fails twice, ask the agent for the workflow state and its failed children and bring them to the table.

When something fails, continued

- **The roast shows an error mentioning 401.** The key was not substituted or is wrong. Create a new key and ask the agent to update the app's environment, keeping every existing variable.
- **RunPod answers 402.** Your credit is gone. Check the console.
- **A generation hangs.** Ask the agent to read the app logs and, for your own endpoint, the endpoint's workers in the RunPod console. Do not resend the same request repeatedly.
- Your agent can print your workflow rows with `list_workspace_workflows`, and if a row has scrolled off, with `panel_request GET /v1/workspaces/<id>/workflows` and params `{"name": "instance.provision"}`.
- When you ask for help, bring: your tool, the step you were on, the ids from `HACKATHON.md`, and the exact error text with your key removed.
- Help: the GalaxyGate table, or the hackathon Discord at <https://discord.com/invite/HP4BhW3hnp>

After the event

- Delete the server from the GalaxyGate panel, or ask your agent to delete it.
- Revoke the `hackathon-app` key in the RunPod console.
- Check that no RunPod endpoint you created still has workers running.